

PARTICIPEZ A UNE COMPETITION KAGGLE !

Rendez-vous sur le site de Kaggle et choisissez la compétition qui vous intéresse. Vous êtes libre de votre choix ! Vous devrez participer à la compétition dans la mesure du possible, ou en tout cas obtenir des résultats mesurables. Vous passerez par toutes les étapes de l'analyse : récupération, nettoyage des données, analyse exploratoire, création de plusieurs modèles et mesure de leurs performances, etc.

FRECHOU Damien,
parcours Ingénieur
Machine Learning

Table des matières

A. Introduction.....	2
I. Contexte	2
a) Intitulé du projet 8	2
b) Compétition Kaggle : IA Génératives.....	2
II. Compétition: « I’m Something of a Painter Myself »	3
a) Enoncé	3
b) Evaluation	3
B. Les IA génératives : GAN	4
I. GAN : Generative Adversial Network	4
a) Principe.....	4
b) Loss function.....	5
c) Architecture CNN.....	6
C. Implémentation et analyse	7
I. Implémentation.....	7
a) Exemple d’un notebook tutoriel: CycleGAN.....	7
b) Choix d’un CNN.....	8
II. Analyse et résultats	10
a) Résultats du modèle de contrôle CycleGAN.....	10
c) Résultats des différents GANs	11
III. Conclusion	13
Bibliographie.....	14
Articles.....	14
Codes	14
Ressources diverses.....	14

A. Introduction

I. Contexte

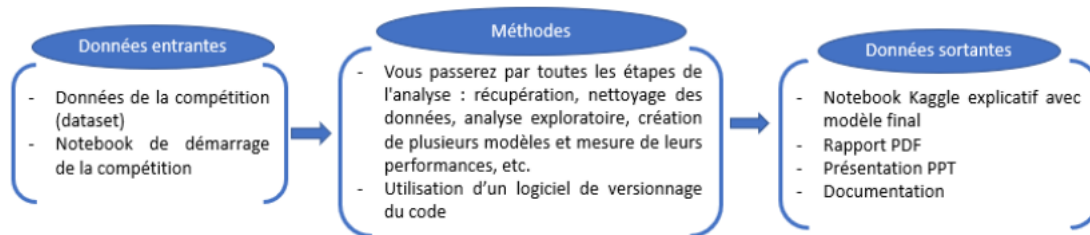
a) Intitulé du projet 8

L'objectif du projet 8 est de participer à une compétition en Data Sciences sur la plateforme web Kaggle :

[Kaggle: Your Machine Learning and Data Science Community](https://www.kaggle.com/).

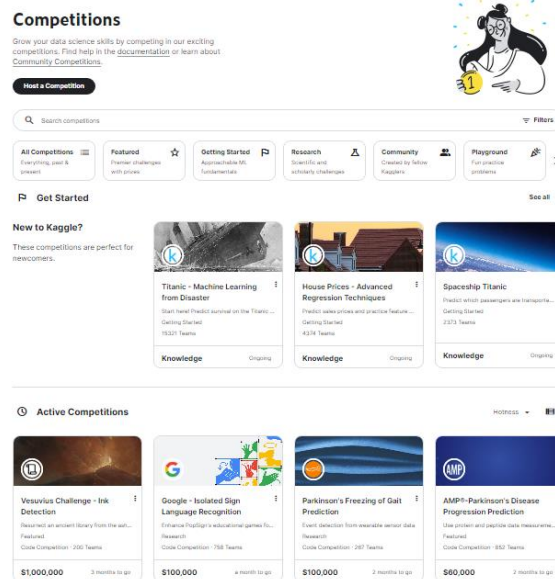
Cette plateforme permet notamment de réaliser des notebooks en code Python ou R qui seront exécutés sur des serveurs en lignes dédiées à la sciences des données.

A partir des données fournies par la problématique de la compétition, il va falloir réaliser différents livrables, comme montrés sur le schéma suivant.



b) Compétition Kaggle : IA Génératives

Sur la plateforme de Kaggle, on peut visualiser un certain nombre de compétitions disponibles à l'étude :



Elles répondent en général à une problématique bien précise, et sont parfois instanciées par des groupes privés qui récompensent les meilleurs candidats classés.

Les compétitions ont normalement une durée de vie limitée d'environ 3 mois. Lorsque leur date de fin de disponibilité n'est pas atteinte, elles sont considérées actives. C'est ce type de compétition qui nous intéresse. Parmi celles-ci, il y a les « Getting Started competitions », des compétitions à but éducatif qui sont en permanence renouvelées. Etant intéressé par l'analyse d'images, l'une d'elles a attiré mon attention, et ce notamment car elle permet de comprendre un domaine du machine learning en plein essor : les intelligences artificielles génératives. Les IA génératives ont la capacité de générer des données précises et élaborées à partir de requêtes qui peuvent être très restreintes. Elles sont de plus en plus mise en avant notamment pour leur capacité à générer des images originales (cf : Images ci-dessous) ou du texte comme l'IA ChatGPT, qui est l'outil le plus médiatisé de ce début d'année 2023.



Images générées par les IA Midjourney et Dall-E à partir de mots clés restreints.

Pour comprendre le principe de base de ces IA, j'ai chois de m'inscrire à la compétition « I'm Something of a Painter Myself » permettant de coder de zéro ce type d'IA.

II. Compétition: « I'm Something of a Painter Myself »

a) Enoncé

Les détails de la compétition se trouvent au lien suivant : <https://www.kaggle.com/competitions/gan-getting-started/overview/description>

Les attendus de cette compétition sont les suivant :

- Générer entre 7000 et 10000 images avec le style du peintre Monet.
- Les images générées doivent être au format 256x256 RGB et zippées dans un fichier images.zip.
- Le temps d'exécution des notebooks soumis doit être <5h si CPU/GPU et <3h si TPU

Pour réaliser ces images, il faudra créer un modèle de Deep Learning de type GAN permettant de générer à partir de rien d ou de photos réelles des peintures au style Monet.

Pour pouvoir réaliser ce modèle, on dispose des données d'entraînement suivantes :

- 300 peintures de Monet en 256x256 JPEG ou TFRecord
- 7028 photos réelles en 256x256 JPEG ou TFRecord

Un notebook permettant de bien démarrer est fournit dans la description. Il a été réalisé par Amy Jang, et ce trouve au lien suivant :

<https://www.kaggle.com/amyjang/monet-cyclegan-tutorial>

b) Evaluation

La métrique d'évaluation utilisée par kaggle pour savoir si l'image générée est considérée bonne est la MiFD: Memorization-informed Fréchet Inception Distance. C'est une mesure de distance entre les échantillons d'images générées et des échantillons réels. Les grandeurs comparées proviennent des caractéristiques relevées des couches profondes d'un réseau pré-entraîné sur un plus grand dataset (ici Inception sur ImageNet) .

Plus la MiFD est petite, meilleur sont les images générées.

Nous ne nous intéresserons pas aux détails de cette métrique sont consignés sur la page suivante : <https://www.kaggle.com/competitions/gan-getting-started/overview/evaluation>

Lorsqu'un notebook génère le fichier zip demandé par la compétition, on soumet directement le notebook ou ce fichier à Kaggle, qui se chargera d'effectuer le calcul de MFID et de nous retourner le score généré ainsi que notre classement sur la compétition.

B. Les IA génératives : GAN

I. GAN : Generative Adversarial Network

Les Generative Adversarial Network (GAN) sont un type de réseau de neurones qui ont connu une grande popularité dans le domaine de l'apprentissage machine, notamment dans le domaine de la vision par ordinateur. Les GAN ont été introduits pour la première fois par Ian Goodfellow et ses collègues en 2014, et depuis lors, ils ont été utilisés pour générer des images, des vidéos et même de la musique.

Dans cette étude, nous ne nous intéresserons spécifiquement qu'aux IA générant des images.

a) Principe

Les GAN sont composés de deux réseaux de neurones qui travaillent ensemble de manière antagoniste : un générateur et un discriminateur. Le générateur prend en entrée un vecteur aléatoire et génère des données synthétiques qui ressemblent à des données réelles. Le discriminateur, quant à lui, prend en entrée à la fois des données réelles et synthétiques et essaie de les distinguer. L'objectif du générateur est de tromper le discriminateur en générant des données synthétiques qui sont indiscernables des données réelles, tandis que l'objectif du discriminateur est de classer correctement les données en entrée comme étant réelles ou synthétiques.

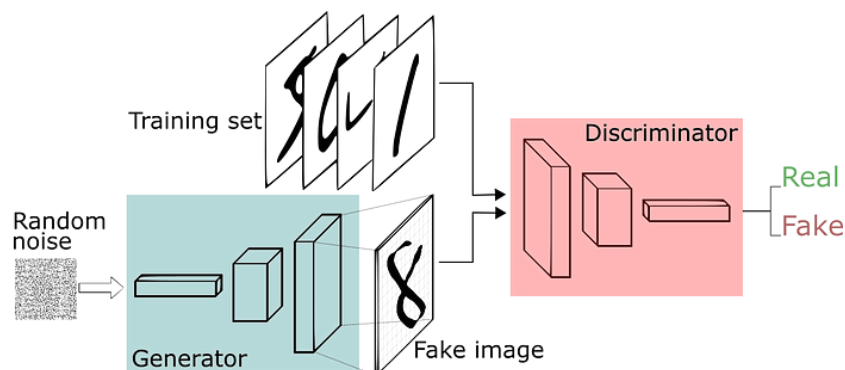


Illustration d'un GAN générant des chiffres dans une image.

<https://towardsdatascience.com/generative-adversarial-networks-explained-34472718707a>

Le processus de formation des GAN est divisé en deux phases. Dans la première phase, le générateur est entraîné pour générer des données synthétiques qui ressemblent aux données réelles. Dans la deuxième phase, le discriminateur est entraîné pour distinguer entre les données réelles et synthétiques. Le processus d'entraînement est répété jusqu'à ce que le générateur puisse générer des données synthétiques qui sont indiscernables des données réelles, et que le discriminateur ne puisse pas faire de distinctions entre les données réelles et synthétiques.

Au fur et à mesure que le générateur est entraîné, il génère des données synthétiques qui ressemblent de plus en plus à des données réelles, et le discriminateur est également entraîné pour être de plus en plus précis dans sa classification.

Une back-propagation propre aux réseaux de neurones artificiels permet de réaliser cet apprentissage de manière efficace, à l'aide d'une fonction de perte propre à chaque réseau.

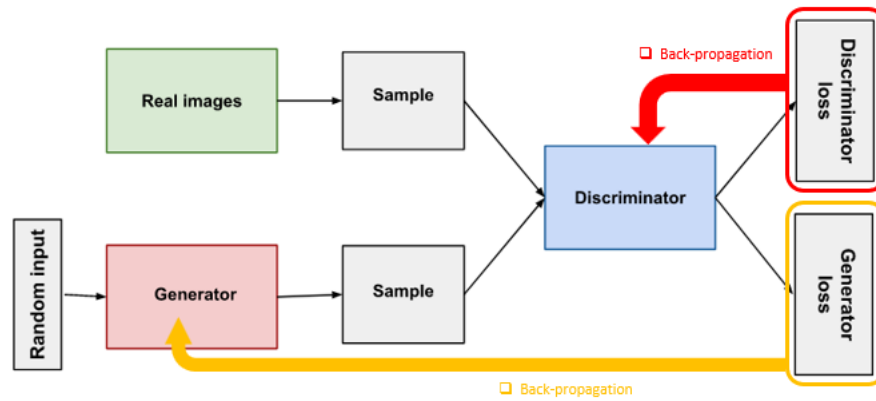


Image représentative de la back-propagation dans un GAN.

https://developers.google.com/machine-learning/gan/gan_structure

b) Loss function

Les GAN tentent de reproduire une distribution de probabilité. Ils doivent donc utiliser des fonctions de perte qui reflètent la distance entre la distribution des données générées par le GAN et la distribution des données réelles.

En pratique, on utilise l’Espérance mathématique du logarithme des sorties générées. Elle se présente généralement sous la forme suivante :

$$\min_{\theta_g} \max_{\theta_d} \left[\underbrace{\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x)}_{\text{Discriminator output for real data } x} + \underbrace{\mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))}_{\text{Discriminator output for generated fake data } G(z)} \right]$$

[Generative Adversarial Networks — Explained | by Rohith Gandhi | Towards Data Science](#)

X: image réelle

z: vecteur de bruit aléatoire

D: Sortie Discriminator

G: Sortie Generator

E : Espérance mathématique

Après certaines simplifications, on peut calculer les gradients des fonctions de pertes avec les deux formules respectives suivantes :

Gradient Generator :

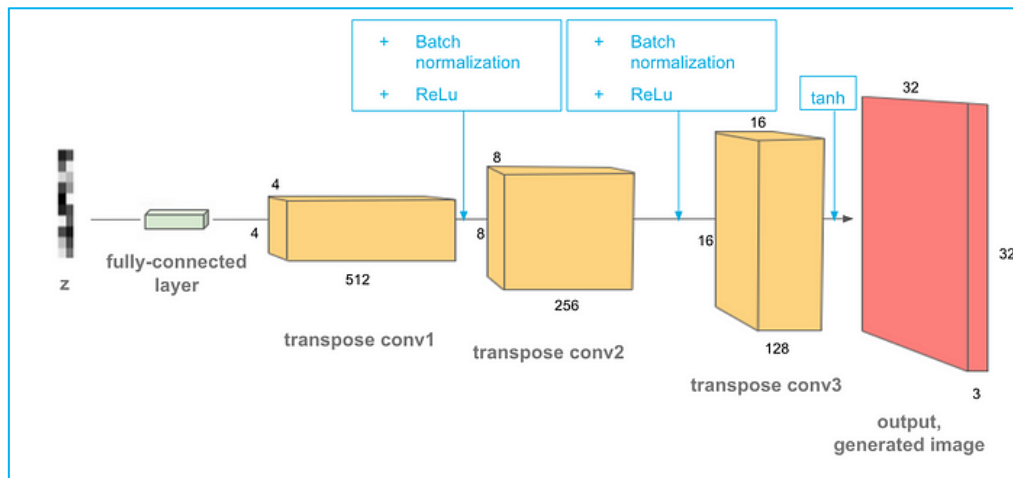
$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z^{(i)})))$$

Gradient Discriminator :

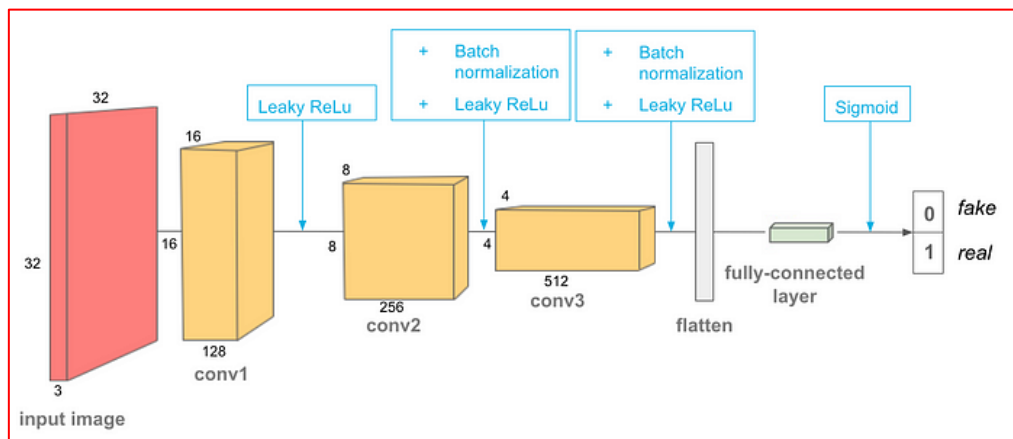
$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log(1 - D(G(z^{(i)}))) \right]$$

c) Architecture CNN

Les architectures de base des Discriminateur et Générateur sont en général celles de réseaux convolutifs :



Architecture générale du Générateur



Architecture générale du Discriminateur

<https://yannicksergeobam.medium.com/g%C3%A9nerer-les-images-%C3%A0-laide-des-gans-avec-tensorflow-6c761311929d>

Le générateur prend en entrée un vecteur de bruit aléatoire, qu'il redimensionne dans un espace 3D, permettant ensuite l'usage de chaîne de filtre de convolution profond pour récupérer des caractéristiques particulières d'images, suivi par des unités de Pooling pour réduire et condenser l'information des différents tenseurs. Le générateur va ainsi réduire au fur et à mesure la profondeur des filtres, jusqu'à obtenir un tensor de la taille de l'image à générer.

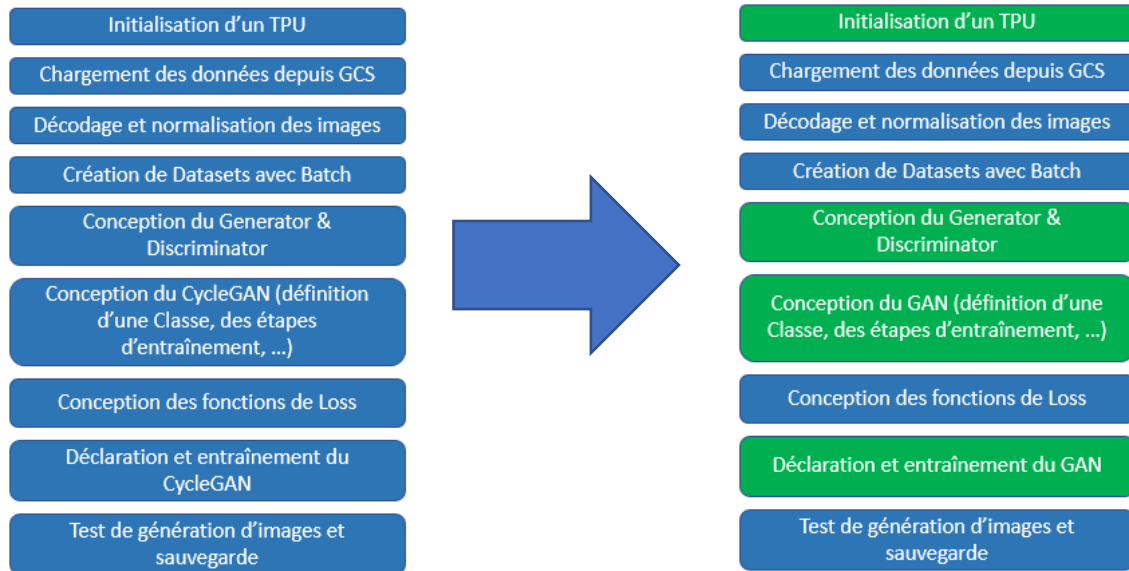
Le discriminateur fonctionne à l'opposé, comme il prend en entrée une image dont il cherchera l'information qui y est contenu en augmentant petit à petit la profondeur de ces filtres. A la fin, on aura une sortie dense de taille 1D reflétant l'appartenance ou non à une seule classe d'objet : image réelle ou fausse. C'est un classifieur.

C. Implémentation et analyse

I. Implémentation

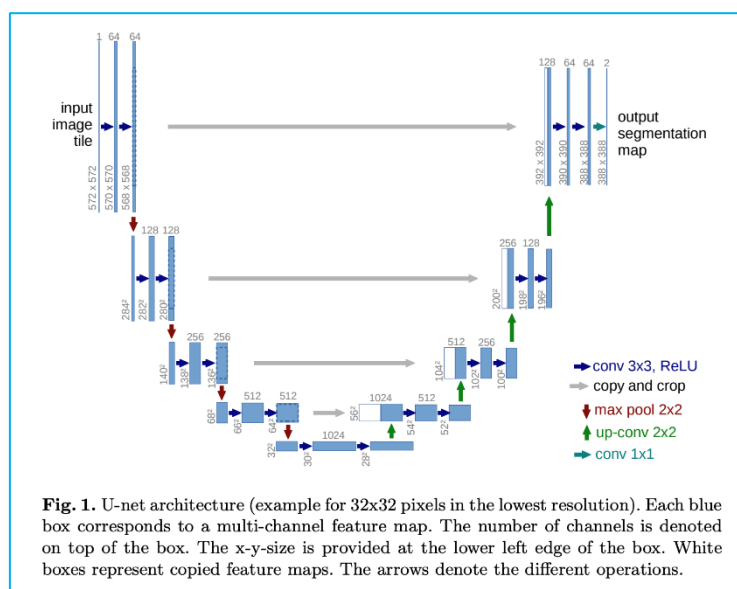
a) Exemple d'un notebook tutoriel: CycleGAN

Pour implémenter ce type de réseau, le notebook fournit pour bien démarrer la compétition est agencé de sorte qu'il soit facile de réaliser certaines étapes. Une description de celles-ci avec celles qui seront à modifier s'illustrent par l'image suivante :



A gauche les étapes du notebook de démarrage, à droite celles que l'on effectuera.

Le notebook de démarrage utilise une structure de GAN particulière appelée CycleGAN qui consiste à transformer une image existante pour qu'elle adopte un style donné. Ici, le style du peintre Monet. L'architecture du générateur est assez différente, c'est celle d'un U-net :

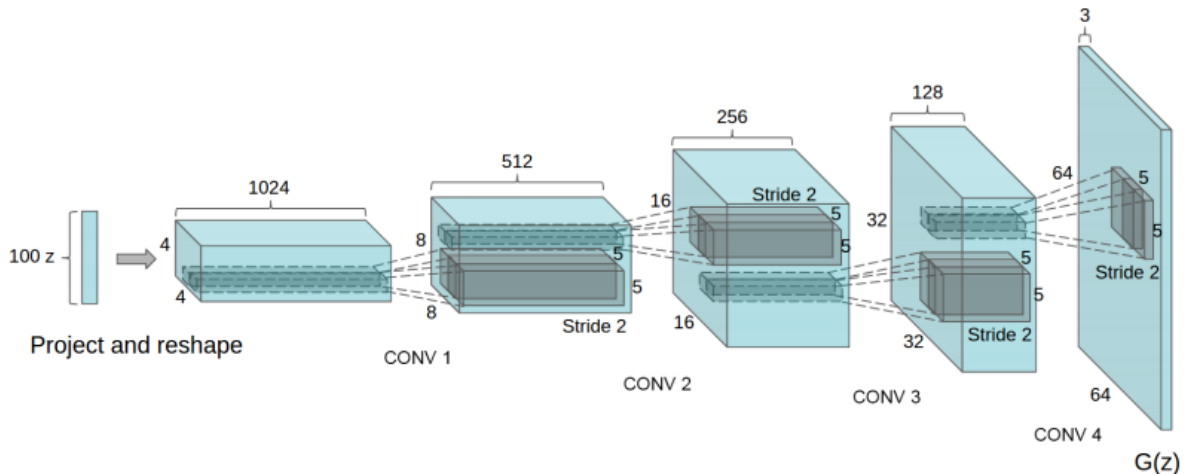


<https://paperswithcode.com/method/u-net>

Elle est composée d'une phase d'encodage de données, suivi d'une phase de décodage de données. Ces deux parties sont reliées par moment par des « skip layers » consistant à ramener partiellement des données caractéristiques de l'encodage directement vers le décodeur, pour garder certaines informations. En effet, il faut pouvoir garder la structure d'origine de l'image d'entrée.

b) Choix d'un CNN

Dans notre étude, on choisit d'implémenter un Générateur avec une structure plus proche d'un réseau convolutionnel classique.



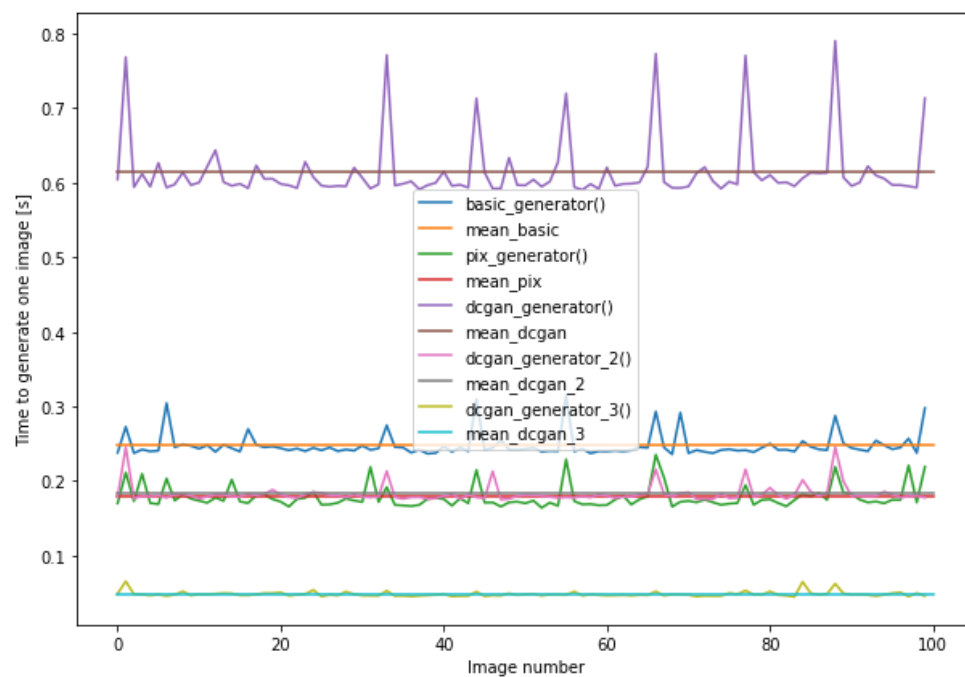
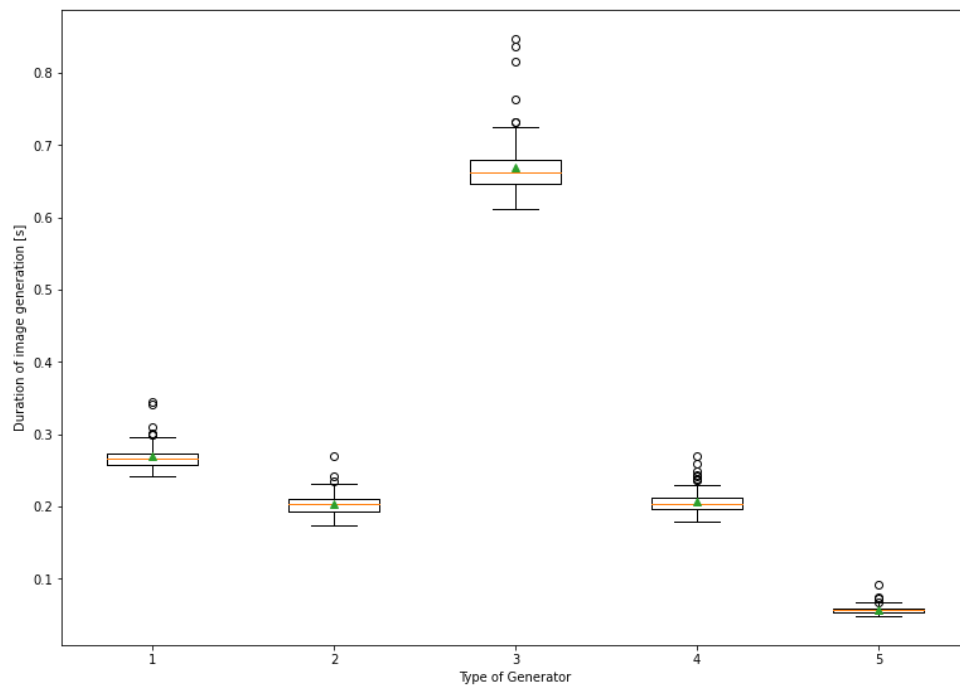
<http://bamos.github.io/2016/08/09/deep-completion/>

En particulier, après avoir tenter de réaliser un réseau proche du cycleGAN du tutoriel, en enlevant les « skip layers », on verra plus tard que ce type de modèle ne semble pas assez adapté pour la génération d'images à partir d'un simple vecteur de bruit.

Mais plusieurs modèles ont été élaborés dans le but d'essayer un maximum de possibilités :

Model	Nombre de paramètres Generator (M)	Nombre de paramètres Discriminator (M)	Type de modèle
basic	13.9	0.67	Convolutionnel
pix	58.9	2.7	U-net sans « skip layers »
dcgan	285.9	278.6	Convolutionnel
dcgn_2	24.6	16.4	Convolutionnel
dcgan_3	7.6	4.1	Convolutionnel

En ayant la contrainte qu'il faille générer une quantité d'images en un temps donné, la première idée a été d'évaluer le temps moyen que prends chaque modèle pour générer 100 images.



Le troisième modèle ayant le plus de paramètre est celui qui génère les images avec un temps le plus long.

Le second modèle (pix), bien qu'ayant 2 fois plus de paramètres que le quatrième, prend quasiment autant de temps que ce dernier.

A l'origine, seuls les deux premiers modèles avaient été implémentés. Mais voyant la difficulté pour obtenir des résultats probant, le besoin de développer des architectures plus « simples » et proches des articles ayant mis en lumière les GAN s'est fait ressentir.

<https://arxiv.org/pdf/1511.06434.pdf>

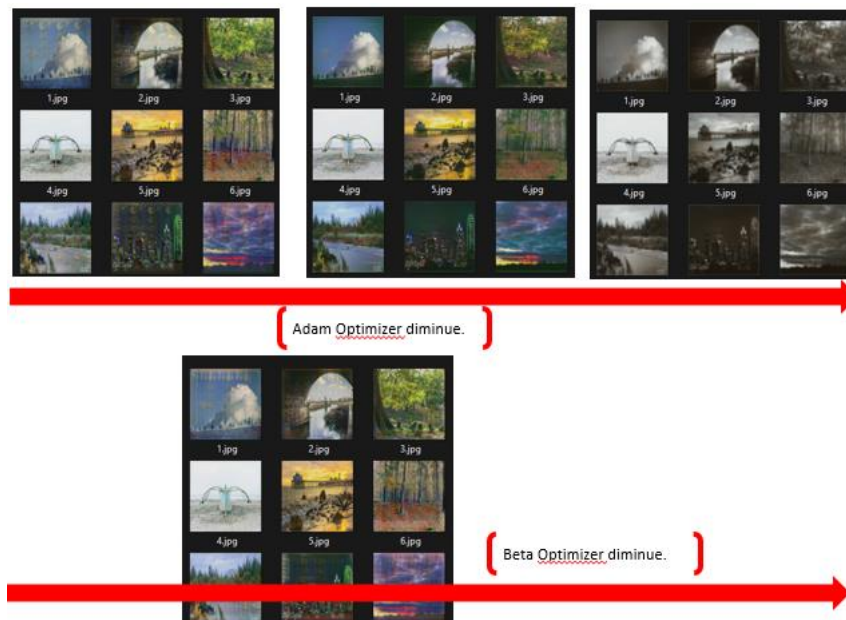
II. Analyse et résultats

a) Résultats du modèle de contrôle CycleGAN

Une étude sur le notebook avec le CycleGAN donne les résultats suivants :

CycleGAN modifié			
Version	Paramètres	Résultats	Score
V1	TPU adam_1= 2e-3 , , beta = 0.5	1h	57.64
V2	CPU	11h39	
V3	TPU, adam_1= 2e-3 au lieu de 2e-4	12 hours: out of range	
V4	TPU, adam_1 = 2e-5	3460s	82.2
V5	TPU, adam_1 = 2e-6	3226s	110
V6	TPU, beta = 0.3	3386s	61.4
V7	TPU, beta = 0.1	3302s	63.5

Entraîné sur 25 epochs, la version par défaut V1 nous donne un score de 57.64 nous classant 53/149, ce qui est un résultat meilleur. En modifiant certains paramètres, notamment dans l'optimizer, Adam il reste difficile d'avoir de meilleurs résultats.



Exemple d'images générées selon l'évolution de certains paramètres.

On notera à quel point l'utilisation de TPU permet d'augmenter grandement la vitesse d'apprentissage. On passe de 11h39 de temps d'exécution sur CPU à 1 avec TPU, pour le même notebook.

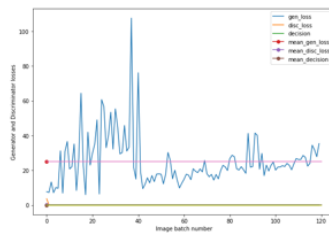
L'objectif par la suite serait d'atteindre voir de dépasser ces scores avec un GAN classique.

c) Résultats des différents GANs

Les résultats obtenus avec les GAN sont assez incertains, selon les modèles, la quantité d'epoch, etc..

Voici quelques exemples :

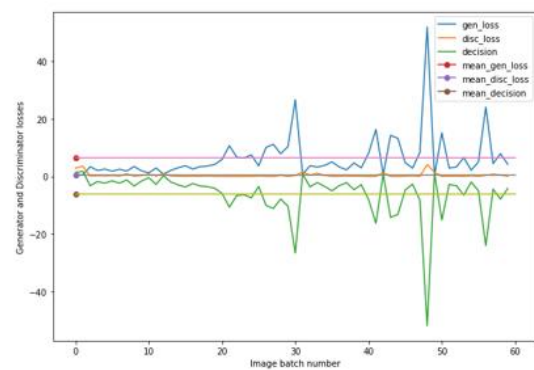
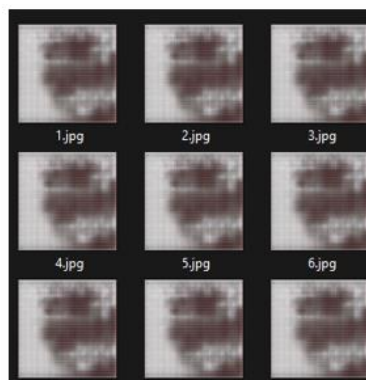
Modèle Basic



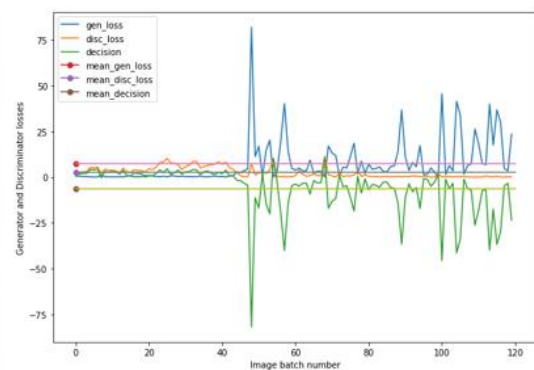
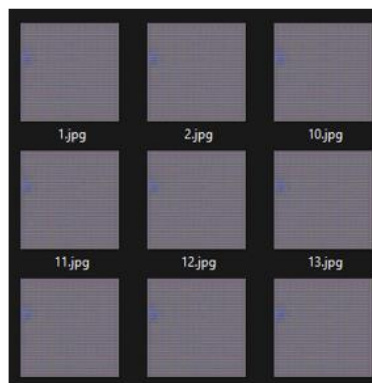
- ❑ V47
- ❑ basic
- ❑ iGAN = 1
- ❑ 120 epochs
- ❑ CPU
- ❑ bs=1
- ❑ 1s/step→450-500s/ep
- ❑ 42629.7s

Modèle pix

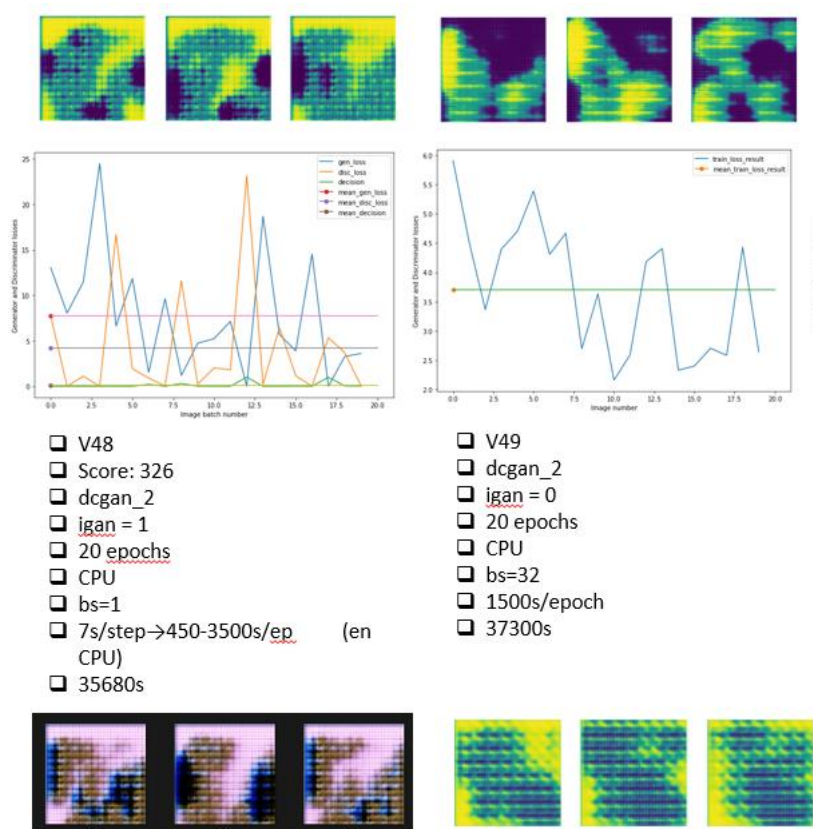
- ❑ 348
- ❑ V22: (g_img4)
- ❑ CPU 60 epochs
- ❑ Batch_size 12
- ❑ iGAN=1
- ❑ 15788s = 4h23



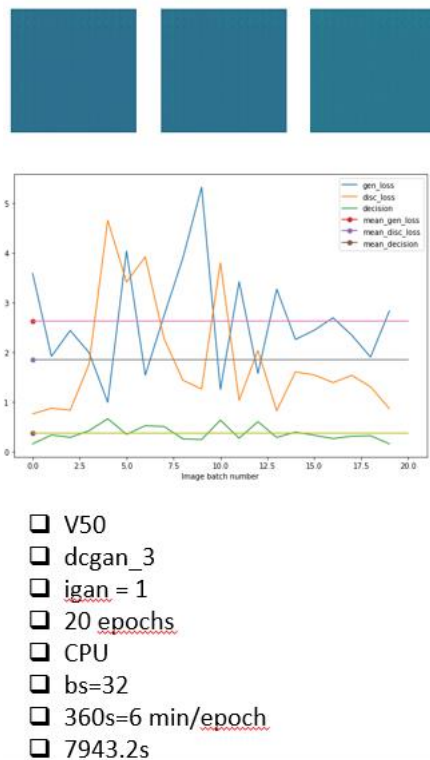
- ❑ 531
- ❑ V35: (g_img8)
- ❑ CPU 120 epochs
- ❑ Batch_size 32
- ❑ iGAN=1
- ❑ 29183s = 8h06



Modèle dcgan_2



Modèle dcgan_3



On notera que les courbes de Loss sont extrêmement dynamiques. Peut-être que le nombre d'epochs pour l'entraînement est trop réduit empêchant de visualiser une tendance des courbes à croître ou décroître.

Certains modèles donnent des motifs qui semblent plus complexes que d'autres modèles, malgré moins de paramètres ou d'epochs d'entraînement.

Model	Nombre de paramètres Generator (M)	Nombre de paramètres Discriminator (M)	Version	Score
basic	13.9	0.67	V47	
pix	58.9	2.7	V22/V35	348/531
dcgan	285.9	278.6		
dcgn_2	24.6	16.4	V48/V49	326/340
dcgan_3	7.6	4.1	V50	390

Il a cependant été difficile de réaliser des entraînements plus longs car il est parfois compliqué de gérer les paramètres des TPUs lorsqu'on élabore soi-même la boucle des étapes d'entraînement d'un modèle.

Ainsi, certaines implémentations sont réalisées avec des calculs de fonctions de Loss et de gradients qui peuvent varier. On en a réalisé 3 différentes (les trois classes de GAN du notebook).

III. Conclusion

Il est difficile de choisir une bonne architecture sans avoir la possibilité de tester un certain nombre de paramètres rapidement. En particulier, même si ce n'était pas facile d'utiliser les TPUS, lorsqu'ils sont disponibles, il faut parfois attendre près de 3h pour que le notebook s'exécute. Autrement, avec un CPU, le temps d'entraînement peut se révéler extrêmement long.

On retiendra les points suivants pour cette étude :

- Utilisation de TPUs pour l'accélération du temps d'apprentissage par parallélisation des calculs mais implémentation fastidieuse lorsqu'il faut créer soi-même la boucle d'entraînement du modèle.
- Les TPUs ne sont pas toujours aisément disponibles. Création d'un notebook d'entraînement continue des modèles.
- Les modèles les plus complexes ne sont pas nécessairement les plus performants. Ils doivent être adaptés au type de problème rencontrés.
- La taille du dataset est relativement restreinte.
- Possibilité d'utiliser la MFID comme métrique pour l'apprentissage ?
- Manque de temps pour effectuer plus d'essai d'apprentissage.
- Question de société : droits d'auteur et étiquette.

Bibliographie

Articles

- [1] :

Codes

- [2] :

Ressources diverses

- [1] :
-